A2 – S4 – 2025/2026 – II.2415
Advanced Algorithms and Programming
Semester Project

LAB_6

Basics of Graph Structures

Session Objectives

By the end of this session, you will be able to:

1. Define and implement graph structures using adjacency matrix and adjacency list representations
2. Implement basic graph traversal algorithms (DFS and BFS) for social network analysis
3. Analyze graph properties (degree, connectivity, path finding)
4. Apply graph traversals to solve real social network problems (friend recommendations, connection paths)
5. Compare and choose appropriate graph representations based on use cases

Core Exercises

Exercise 1: Graph Representations for Social Networks

Objective: Implement both adjacency matrix and adjacency list representations for a social network graph where users are nodes and friendships are edges.

Social networks naturally model users as vertices and friendships as edges. Understanding both graph representations is crucial for optimizing different operations.

Requirements:

1. Create a SocialGraph structure with two internal representations: Adjacency Matrix - Adjacency Linked List.
2. Implement core graph operations for each representations:

Operation	**Adjacency Matrix**	**Adjacency List**
add_friendship(u, v)	Add edge between users u and v	Add edge between users u and v
remove_friendship(u, v)	Remove edge between users	Remove edge between users
are_friends(u, v)	Check if u and v are friends	Check if u and v are friends
get_friends(u)	Get all friends of user u	Get all friends of user u
get_degree(u)	Return number of friends of user u	Return number of friends of user u
get_num_users()	Return total vertices	Return total vertices
get_num_edges()	Return total edges	Return total edges

3. Implement graph property calculations:
    - is_complete_graph(): Check if every user is friends with every other user
    - graph_density(): Calculate edge density = 2|E|/(|V|(|V|-1)) for undirected graphs

- degree_distribution(): Return dictionary mapping degree to count of users

4. Implement conversion between representations:
   - matrix_to_list(): Convert adjacency matrix to adjacency list
   - list_to_matrix(): Convert adjacency list to adjacency matrix

Complexity Analysis Questions:

1. What is the time complexity of are_friends(u, v) in each representation? Prove it.
2. For a social network with 1 billion users where average user has 150 friends, which representation is more space-efficient? Calculate the exact memory difference.
3. If you need to frequently add/remove friendships, which representation is better? Why?

Exercise 2: Graph Traversals - DFS for Social Network Analysis

Objective: Implement DFS to explore social connections, find connected components, and detect friendship circles.

Remember that DFS is useful for:

- Finding all users in a connected component (friendship circle)
- Detecting if a path exists between two users
- Finding connected components in the network
- Detecting isolated users

Requirements:

1. Implement DFS traversals:
   Part A - Recursive DFS: dfs_recursive(start_user): Return list of users in DFS order starting from start_user
   Part B - Iterative DFS using stack: dfs_iterative(start_user): Return list of users in DFS order using explicit stack
   Part C - Find all connected components: find_connected_components(): Return list of lists, each inner list is a connected component
   Part D - Check if graph is connected: is_connected(): Return True if graph has exactly one connected component
   Part E - Find if path exists between two users: has_path(start_user, target_user): Return True if there's a path between start and target
   Part F: Find path between users: find_path(start_user, target_user): Return list of users representing path from start to target
2. Implement traversal-based analytics:
   - get_connected_components_sizes(): Return list of sizes of each connected component
   - find_largest_component(): Return the largest friendship circle
   - find_isolated_users(): Return list of users with degree 0 (no friends)

Complexity Analysis Questions:

1. What is the time and space complexity of DFS on an adjacency list representation? On adjacency matrix?
2. Why might recursive DFS cause problems in a social network with 1 million users? How would you solve it?

Exercise 3: Graph Traversals - BFS for Shortest Path Analysis

Objective: Implement BFS to find shortest paths, degrees of separation, and perform level-order exploration of social networks.

Remember that BFS is useful for:

- Finding shortest path between users (minimum degrees of separation) [n.b. Degree of separation is a measure of the social distance between two people in a network, defined as the minimum number of edges (friendship connections) that must be traversed to connect one person to another].
- Computing "Six Degrees of Separation" type metrics
- Finding all friends within k degrees of separation
- Level-by-level exploration of social circles
- Friend recommendations based on friends-of-friends

Requirements:

1. Implement DFS traversals:
   Part A - Basic BFS using queue: bfs(self, start_user): Return list of users in BFS order starting from start_user
   Part B - BFS with distance tracking: bfs_with_distances(self, start_user): Return dictionary of {user: distance} from start_user
   Part C - Find shortest path between users: shortest_path(self, start_user, target_user): Return list representing shortest path between users
   Part D - Find degrees of separation: degrees_of_separation(self, start_user, target_user): Return number of edges in shortest path (-1 if no path)
   Part E - Find all users within k hops: friends_within_k_hops(self, start_user, k): Return set of users reachable within k edges from start_user
2. Implement traversal-based analytics:
   - compute_average_degrees_of_separation(): Average distance between all pairs of connected users
   - get_distance_distribution(start_user): Distribution of distances from a user to all others
   - recommend_friends(start_user, max_recommendations=5): Friend recommendation based on friends-of-friends not already friends

Complexity Analysis Questions:

1. What is the time and space complexity of BFS? How does it compare to DFS?
2. In an unweighted social network, why is BFS guaranteed to find the shortest path?
3. For finding "six degrees of separation" between two users, would you use BFS or DFS? Why?
4. In a social network, would you use DFS or BFS to find the shortest friendship chain? Why?

Final Integration Question

Feature Design: "Social Network Friend Recommendation & Analysis Engine"

Design a comprehensive friend recommendation and social network analysis engine that combines all graph concepts.

Components:

1. Graph Representation (Exercise 1): Store user friendships with optimal representation
2. DFS Applications (Exercise 2): Find connected components and detect isolated users
3. BFS Applications (Exercise 3): Compute degrees of separation and friend recommendations

Requirements:

The system must support the following features for a social network with 1,000,000 users and average degree 150:

Feature	**Required Operation**
Friend suggestion	Find friends-of-friends not already connected
Degrees of separation	Shortest path between two users
Find isolated users	Users with no friends
Detect friendship circles	Find all connected components
Six degrees verification	BFS from a user to compute distance distribution

Questions to Answer:

1. Representation Choice:
   - Which representation (adjacency matrix vs adjacency list) would you choose for this social network? Justify with memory calculations.
   - Calculate the exact memory usage for each representation with 1M users and average degree 150.
2. Traversal Selection:
   - To find the shortest friendship chain between two users, which traversal do you use and why?
   - To find all users within 3 degrees of a target user (friend suggestions), which traversal is most appropriate?
   - To detect all isolated users, which approach would you use?
3. Scalability Considerations:
   - How would you handle BFS on a graph with 1M users where the target might be 6 degrees away?
   - Describe an iterative approach to avoid recursion limits in DFS.
   - What is the maximum number of operations needed to recommend friends for a user with degree 150?
4. Performance Analysis:
   - Calculate the time complexity of recommending friends for a user with degree d in a graph with n users.
   - How many operations are needed to find all connected components using DFS?

Example Scenario:
A user requests:

1. Find 10 friend recommendations
2. Show the shortest path to a specific friend (3 degrees away)
3. Find which connected component they belong to and its size

For each task, specify:

- Which representation you use
- Which traversal algorithm
- Expected time complexity
- Estimated number of operations

General Instructions

Repository Setup
Create branch: LAB _06_BasicsOfGraphs

File Structure:
Follow the same structure of LAB _05

Git Requirements:
Same as for LAB _05

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 10	**Note: 10**
• Each Team (of number BB) Upload at the **end of the session** a very short PDF document and name it **TBB_LAB6_AES.pdf**, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	• You finish all the implementation of all exercises, • You finalize the previous report and give it a new name **TBB_LAB6_BFM.pdf** and upload it before **Friday midnight of the same week**, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.